2025 – 7th May, 23:55PM

ISE102 Software Engineering Assessment Three

Name: Khaled Elmasri

Student ID: A00173074

Email: Khaled.Elmasri@Student.Torrens.Edu.Au

Assessment Description

Audio-Visual Presentation Video of the Final Software Solution along with supporting UML diagrams

This assessment tasked us to continue building the software we started in assessment two for a major bank in NSW Australia inputting a new deposit and withdraw method using the "Object Oriented Programming" style to allow for better scalability and modularity to further improve any future programs or software we endeavor to create in the future. We were also tasked with creating UML diagrams that relate to the created software for a higher-level overview of what was improved upon as well as providing justification for them.

(All UML diagrams in this document were created by Khaled Elmasri using Lucid Chart)

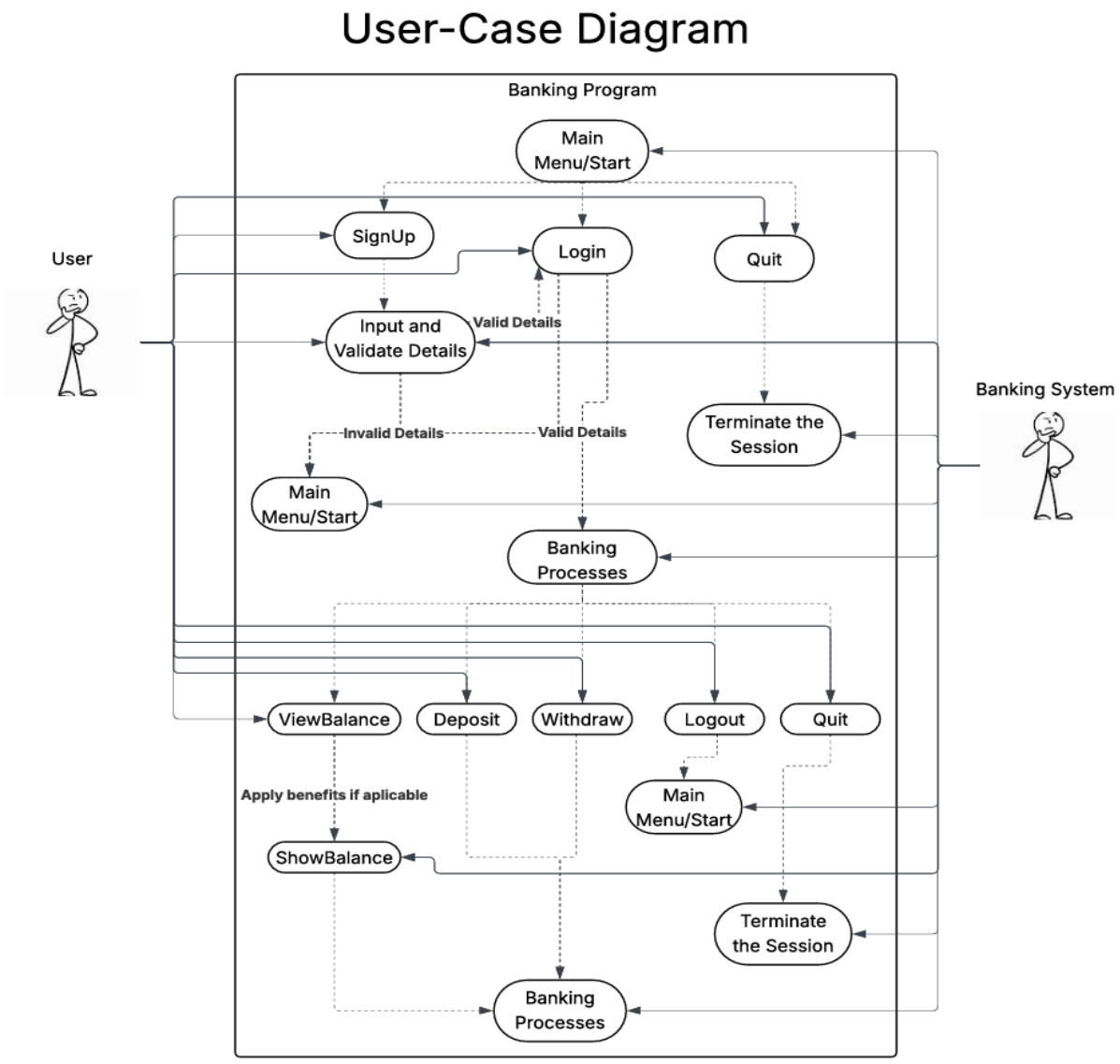
Table Of Contents

Unified Modeling Language (UML) Diagrams

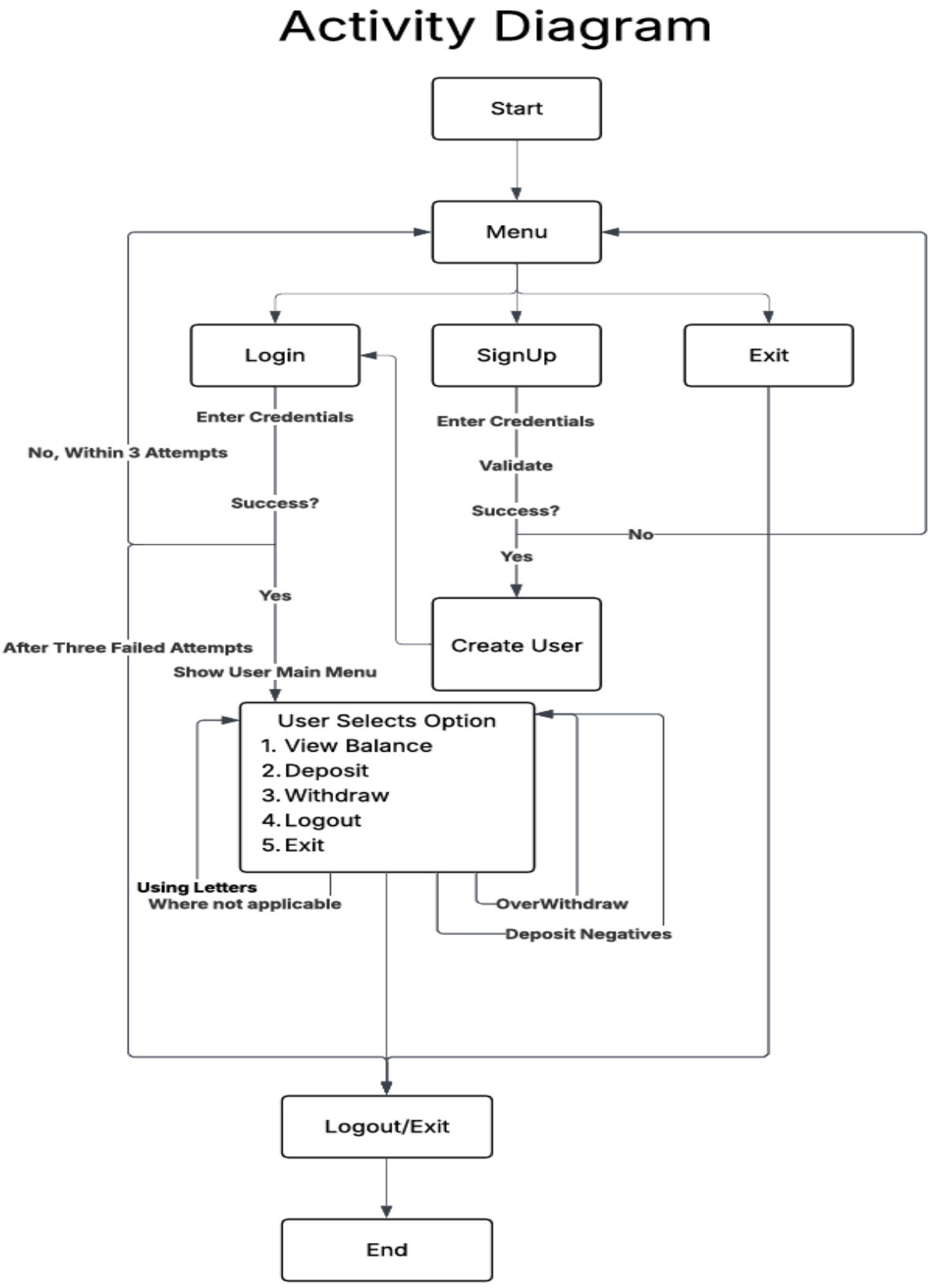
Why use UML Diagrams?

“Unified Modeling Language (UML) is a standardized visual modeling language that is a versatile, flexible, and user-friendly method for visualizing a system’s design” (GeeksForGeeks, 2025). Furthermore, UML diagrams allow system items to be built, visualized and specified as documents for better understanding for stakeholders and users that were not involved in the creation process.

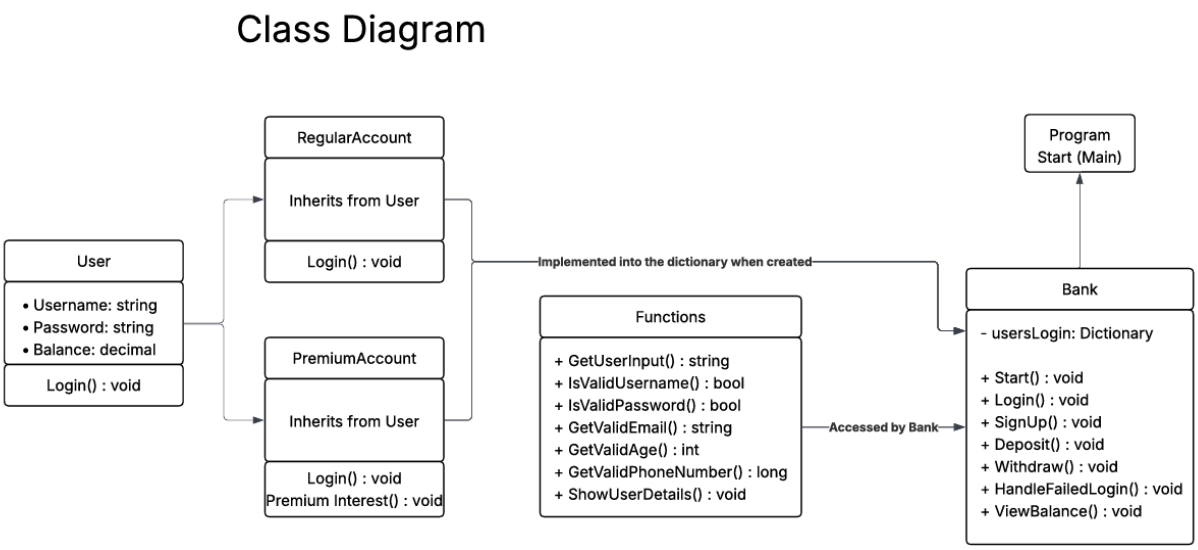
User-Case Diagram



Activity Diagram



Class Diagram



UML Diagrams Addition Information

Program Specification Design (PSD)

PSD refers to the process of defining and documenting the requirements and design of a system. UML plays a significant role in this phase allowing developers, stakeholders and teams to understand how the system should behave and be structured (Refer to the activity diagram). UML diagrams are used to clarify requirements, provide high-level overviews, identify system components and interfaces, process flow and behavior and define data flows and states.

- **Clarify Requirements:** User-case diagrams help gather and specify functional requirements. For example, this assessment tasked us to create a banking system that at the bare minimum defined “Login”, “Signup”, “Withdraw” and “Deposit” as core operations that users can perform.
- **High-Level Overview:** Class and Component Diagrams give a high-level view of how the system is structured (as seen with the Class Diagram). With the above example we can see that it showcases key classes, objects and components in the system as well as their relationships. Helping developers understand the architecture of the system before starting to write code.
- **Identifying System Components and Interfaces:** Component and Class Diagrams define the structure of systems and components and how they interact. It is especially useful for larger, more complex systems to ensure components meet a well-defined purpose and is cohesive.
- **Process Flow and Behavior:** Activity and Sequence Diagrams showcase the workflow and interactions with the program/system. An activity diagram is used to map out the flow of

activities, for example mapping out the process of logging into a bank from start to finish. This helps visualize the steps and decisions involved in each use case.

- **Defining Data Flows and States:** State Diagrams and others similar can be used to model how an object or system's state changes in response to events. For instance, a state diagram can depict the lifecycle of a user's account (active, suspended, closed) based on interactions like logging in, depositing or withdrawing.

Program Verification

This phase ensures that the system or program behaves as expected and that all requirements have been implemented correctly.

- **Validating Requirements Against Designs:** User-case Diagrams can clarify if the design reflects all the required functionality. For example, if a "Transfer Funds" operation is a requirement for a banking system, The user-case diagram would confirm whether it is included and how it's related to users and the rest of the system.
- **Cross Referencing Between Diagrams:** UML Diagrams like Class, Sequence and Activity Diagrams can be cross referenced to ensure that design and implementation align. When related to a Sequence Diagram, this can verify interactions between objects follow the correct order and that objects collaborate as specified in the Class Diagram.
- **Ensuring Consistency:** When creating multiple UML Diagrams, it is important that all diagrams relate to each other to show that the system is logically sound. For example, a State Diagram for a user's account should align with the transactions described in the Sequence Diagram (e.g. how a "suspended" state can change to "active" after payment).
- **Behavioral Validation:** Activity and Sequence Diagrams can be used to verify that the behavior of the system follows the expected process flow. If the diagram shows that the process goes through a certain state before completion, the actual system should match this flow during testing.

Program Validation

Ensuring that the system meets the user needs and requirements is crucial, during this phase the key question is “Are we building the right system?”.

- **Ensuring User Interaction is Correct:** User-case Diagrams validate whether the system is performing as intended to what the user expects to achieve. For example, if the user cannot withdraw money due to an error or it does something completely different than the User-case Diagram, then the system may not meet user requirements, prompting corrections.
- **Testing Real-Life Scenarios:** Activity and Sequence Diagrams simulate user interactions and scenarios. Most of the time these diagrams provide step-by-step visual representation of the system. By mapping out real-life actions (e.g., user logging in, performing a transfer), these diagrams allow testers to ensure the system performs as expected.
- **Confirming Data Integrity and State Transitions:** State Diagrams can be validated by ensuring that the transitions between states represent real-world behavior. For example, when a user submits an incorrect password, the system might move to a “Locked” state after three attempts. Validation of such transitions ensures that the system mimics actual use cases.
- **Stimulating Edge Cases:** Activity Diagrams allow developers to simulate not just normal user actions but also edge cases or exceptions. For example, a user might attempt to withdraw more funds than available. An activity diagram will visualize similar scenarios to ensure the system can handle it.

Other UML Diagrams in Similar Software Programming Projects

There are many different types of UML diagrams that can be utilized in design, verification and validation of software projects other than User-case Diagram, Activity Diagram and Class Diagram:

- **Sequence Diagrams:** This diagram describes how objects interact in a particular sequence. They are useful in specifying the order of the method calls between objects, which is essential for understanding the system's behavior (GeeksForGeeks, 2025).
- **State Diagrams:** State Diagrams describe how the system's objects change states in response to events. They are useful for modeling systems with complex state transitions (e.g. vending machines or order processing systems).
- **Component Diagrams:** "A UML deployment diagram is a diagram that shows the configuration of run time processing nodes and the components that live on them" (Visual Paradigm, N.A.). These diagrams show physical components (such as databases, services, etc.) of the system and their relationships. They are useful for systems with multiple subsystems that must work together.
- **Deployment Diagrams:** This type of diagram models the physical deployment of software components, showing how software components are deployed on hardware nodes. This is useful in cloud systems or distributed applications.
- **Package Diagrams:** Finally, this diagram represents the structure of the code showing how classes and components are grouped into packages. They help manage large systems by organized classes logically.

Reference List

APA References:

- GeeksForGeeks. (2025, January 02). *Unified Modeling Language (UML) Diagrams.* Unified Modeling Language (UML) Diagrams | GeeksforGeeks
- GeeksForGeeks. (2025, January 03). *Sequence Diagrams – Unified Modeling Language (UML).* Sequence Diagrams – Unified Modeling Language (UML) | GeeksforGeeks
- Visual paradigm. (N.A.). *What is a Deployment Diagram?* What is Deployment Diagram?